

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

ОТЧЕТ ПО
ЛАБОРАТОРНОЙ РАБОТЕ № 6

**«Расширение возможностей класса: Реализация
многопоточной системы интегрирования
функций. Сравнение последовательного и
многопоточного методов.»**

по курсу
Объектно-ориентированное программирование

Выполнила: Янышина Анастасия Юрьевна
Группа 6203-010302D

Содержание

Титульный лист	1
Содержание	2
<u>Задание 1</u>	3
<u>Задание 2</u>	5
<u>Задание 3</u>	8
<u>Задание 4</u>	15

Задание 1

Для начала добавим в класс `Functions` метода `integrate`, который бы вычислял значение определенного интеграла для произвольной функции в заданных пределах (Рис. 1). Алгоритм метода трапеций был реализован следующим образом: вся область интегрирования разбивается на участки, длина которых соответствует шагу дискретизации, причем последний участок может быть меньше шага, если границы области не делятся нацело на шаг. Для каждого такого участка вычисляется площадь трапеции, образованной значениями функции на его границах, и эти площади суммируются для получения общего значения интеграла.

Теперь требуется проверить корректность работы метода на примере экспоненциальной функции, определив шаг дискретизации, обеспечивающий точность до 7-го знака после запятой. Для этого был реализован цикл, где метод `integrate` вызывался с различными значениями шага: от 1.0 до 0.000001 (Рис. 2).

После запуска программы выяснилось, что для достижения нужной точности при интегрировании экспоненциальной функции на отрезке $[0, 1]$ достаточно шага дискретизации 0.001 (Рис. 3).

```

1 usage
public static double integrate(Function f, double leftX, double rightX, double step){
    if (leftX==rightX) throw new IllegalArgumentException("Левая граница должна быть меньше правой!");
    if (leftX < f.getLeftDomainBorder() || rightX > f.getRightDomainBorder()) {throw new IllegalArgumentException();}
    if (step <= 0) {throw new IllegalArgumentException("Шаг дискретизации должен быть положительным!");}
    double integral = 0.0;
    double currentX = leftX;

    //проходим по всем участкам
    while (currentX < rightX) {
        double nextX = Math.min(currentX + step, rightX);

        //вычисляем значения функции на границах участка
        double fCurrent = f.getFunctionValue(currentX);
        double fNext = f.getFunctionValue(nextX);

        //добавляем площадь трапеции для текущего участка
        integral += (fCurrent + fNext) * (nextX - currentX) / 2.0;

        currentX = nextX;
    }
    return integral;
}

```

Рис. 1

```

public class Main {
    public static void main(String[] args) {

        //создаем экспоненциальную функцию
        Exp expFunction = new Exp();

        //теоретическое значение интеграла exp(x) от 0 до 1 равно (e - 1) ≈ 1.718281828459045
        double theoreticalValue = Math.E - 1;
        System.out.println("Теоретическое значение интеграла: " + (Math.E - 1));

        //подбираем шаг дискретизации для точности до 7 знака
        double[] steps = {1.0, 0.1, 0.01, 0.001, 0.0001, 0.00001, 0.000001};
        System.out.println();
        System.out.println("-----Поиск подходящего шага дискретизации-----");

        for (double step : steps) {
            double calculatedValue = Functions.integrate(expFunction, leftX: 0, rightX: 1, step);
            double error = Math.abs(calculatedValue - theoreticalValue);

            System.out.printf("Шаг: %.8f | Вычислено: %.15f | Погрешность: %.10f\n", step, calculatedValue, error);

            //проверяем, достигнута ли требуемая точность
            if (error < 1e-6) {
                System.out.printf("Требуемая точность достигнута при шаге: "+ step);
                break;
            }
        }
    }
}

```

Рис. 2

```

"C:\Program Files\Java\jdk-21.0.6\bin\java.exe" "-javaagent:C:\Program Files\
Теоретическое значение интеграла: 1.718281828459045

-----Поиск подходящего шага дискретизации-----
Шаг: 1,00000000 | Вычислено: 1,859140914229523 | Погрешность: 0,1408590858
Шаг: 0,10000000 | Вычислено: 1,719713491389314 | Погрешность: 0,0014316629
Шаг: 0,01000000 | Вычислено: 1,718296147450418 | Погрешность: 0,0000143190
Шаг: 0,00100000 | Вычислено: 1,718281971649195 | Погрешность: 0,0000001432
Требуемая точность достигнута при шаге: 0.001
Process finished with exit code 0

```

Рис. 3

Задание 2

Следующее задание уже связано с потоками. Для начала создаём пакет `threads` (Рис. 4). Далее опишем в этом пакете класс `Task`, по сути, контейнер для параметров задания (Рис. 5 и рис. 6).

Теперь нам требуется описать в `Main` метод `nonThread()`, реализующий последовательную версию программы. Создаём объект `Task`, затем устанавливаем количество заданий. В цикле с помощью `Math.random()` генерируем случайные параметры в указанных диапазонах, определяем функцию логарифма с нужными параметрами и вычисляем интеграл, одновременно с этим выводим в консоль все нужные данные в формате, указанном в ТЗ (Рис. 7).

После запуска всё успешно выводится в консоль, и, вроде, никаких ошибок нет (Рис. 8).

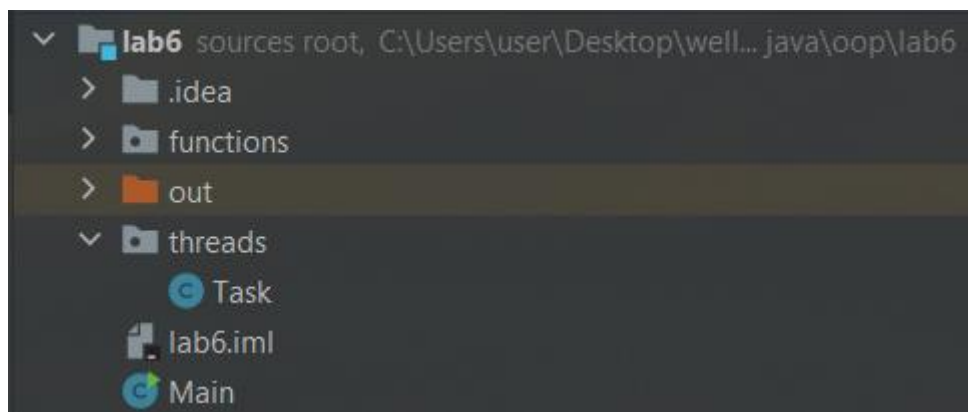


Рис. 4

```

package threads;
import functions.Function;
2 usages
public class Task
{
    2 usages
    private Function function;//интегрируемая функция
    2 usages
    private double leftX;//левая граница интегрирования
    2 usages
    private double rightX;//правая граница интегрирования
    2 usages
    private double discretizationStep;//шаг дискретизации
    3 usages
    private int tasksCount;//количество заданий для выполнения

    //конструктор по умолчанию
    1 usage
    public Task() {}

    //устанавливаем количество заданий
    no usages
    public Task(int tasksCount) {this.tasksCount = tasksCount;}

```

Рис. 5

```

    public Function getFunction() {return function;}
    public void setFunction(Function function) {this.function = function;}
    no usages
    public double getLeftX() {return leftX;}
    no usages
    public void setLeftX(double leftX) {this.leftX = leftX;}
    no usages
    public double getRightX() {return rightX;}
    no usages
    public void setRightX(double rightX) {this.rightX = rightX;}
    no usages
    public double getDiscretizationStep() {return discretizationStep;}
    no usages
    public void setDiscretizationStep(double discretizationStep) {this.discretizationStep = discretizationStep;}
    1 usage
    public int getTasksCount() {return tasksCount;}
    1 usage
    public void setTasksCount(int tasksCount) {this.tasksCount = tasksCount;}
}

```

Рис. 6

```

        System.out.println();
        System.out.println();
        nonThread();
    }

    1 usage
    public static void nonThread() {
        System.out.println("-----Последовательная версия-----");

        //создаём объект задания(?) и устанавливаем количество заданий
        Task task = new Task();
        task.setTasksCount(120);

        for (int i = 0; i < task.getTasksCount(); i++) {
            //генерируем случайные параметры
            double base = 1 + Math.random() * 9; //основание логарифма от 1 до 10
            double left = Math.random() * 100; //левая граница от 0 до 100
            double right = 100 + Math.random() * 100; //правая граница от 100 до 200
            double step = Math.random(); //шаг от 0 до 1

            //создаём функцию логарифма с заданным основанием и выводим изначальные параметры
            Function logFunction = new Log(base);
            System.out.printf("Source %.4f %.4f %.4f\n", left, right, step);

            //вычисляем интеграл и выводим изначальные параметры с результатом
            try {
                double result = Functions.integrate(logFunction, left, right, step);
                System.out.printf("Result %.4f %.4f %.4f %.6f\n", left, right, step, result);
            } catch (Exception e) {System.out.println("Error: " + e.getMessage());}
        }
        System.out.println();
    }
}

```

Рис. 7

```

Main
Source 82,2382 126,5986 0,6677
Result 82,2382 126,5986 0,6677 121,035762
Source 10,5520 104,0436 0,3353
Result 10,5520 104,0436 0,3353 289,436827
Source 35,0256 179,6118 0,1744
Result 35,0256 179,6118 0,1744 1008,980155
Source 9,0878 184,5872 0,1212
Result 9,0878 184,5872 0,1212 407,374493
Source 36,0819 161,9082 0,8347
Result 36,0819 161,9082 0,8347 278,475281
Source 97,9054 110,3562 0,0103
Result 97,9054 110,3562 0,0103 55,126216
Source 51,8245 112,6570 0,9672
Result 51,8245 112,6570 0,9672 122,204011
Source 15,8747 148,6944 0,4738
Result 15,8747 148,6944 0,4738 289,122913
Source 15,7049 151,3823 0,4379
Result 15,7049 151,3823 0,4379 33518,216003
Source 71,2825 197,2163 0,7848
Result 71,2825 197,2163 0,7848 294,145511
Source 60,5758 136,1209 0,5765
Result 60,5758 136,1209 0,5765 527,629369
Source 9,1389 155,9328 0,1793
Result 9,1389 155,9328 0,1793 1397,118682
Source 19,4724 145,7081 0,0828
Result 19,4724 145,7081 0,0828 271,960527

Process finished with exit code 0

```

Рис. 8

Задание 3

Теперь реализуем ещё два класса – `SimpleGenerator` и `SimpleIntegrator` (Рис. 9 и рис. 10), а в главном классе создадим метод `simpleThreads()` (Рис. 11). Уже после первого запуска видим, что программа некорректно работает (Рис. 12). Запустив ещё несколько раз, убеждаемся в этом.

Без синхронизации возникали две основные проблемы:

1. `NullPointerException` – интегратор пытался читать данные до того, как генератор их записывал;
2. Несогласованные данные - интегратор читал частично записанные данные (например, левая граница от одного задания, а правая – от другого).

Для решения этих проблем добавим вспомогательный класс `TaskState`, который будет содержать в себе две переменные – флаги: `dataReady` – показывает, что данные готовы для обработки и `processing` – показывает, что идет процесс обработки, а также счётчики для количества выполненных заданий (Рис. 13). Также исправим классы интегратор и генератор (Рис. 14 и рис. 15).

Генератор теперь работает так: ждет, когда `processing = false` (предыдущее задание обработано); генерирует новые данные и записывает в общий объект; устанавливает `dataReady = true` и `processing = true`; ждет следующего цикла.

Интегратор же теперь: постоянно проверяет, что `dataReady = true` и `processing = true`; если оба условия выполнены, то читает данные и вычисляет интеграл; после обработки устанавливает `dataReady = false` и `processing = false`; переходит к ожиданию следующего задания.

После небольшого исправления в методе `simpleThreads()` и запуска программы видим, что теперь всё работает корректно (Рис. 16 и рис. 17, 18). Для надежности запускаем несколько раз — вроде всё в порядке.

```
public class SimpleGenerator implements Runnable {
    7 usages
    private Task task;
    1 usage
    public SimpleGenerator(Task task) {
        this.task = task;
    }

    @Override
    public void run() {
        for (int i = 0; i < task.getTasksCount(); i++) {
            synchronized (task) {
                //генерируем случайные параметры
                double base = 1 + Math.random() * 9;
                double left = Math.random() * 100;
                double right = 100 + Math.random() * 100;
                double step = Math.random();

                //устанавливаем параметры в задание
                task.setFunction(new Log(base));
                task.setLeftX(left);
                task.setRightX(right);
                task.setDiscretizationStep(step);

                System.out.printf("Source %.4f %.4f %.4f%n", left, right, step);
            }
        }
    }
}
```

Рис. 9

```

1 usage
public class SimpleIntegrator implements Runnable {
    7 usages
    private Task task;
    1 usage
    public SimpleIntegrator(Task task) {
        this.task = task;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < task.getTasksCount(); i++) {
                synchronized (task) {
                    //получаем параметры задания
                    double left = task.getLeftX();
                    double right = task.getRightX();
                    double step = task.getDiscretizationStep();

                    //вычисляем интеграл
                    double result = Functions.integrate(task.getFunction(), left, right, step);
                    System.out.printf("Result %.4f %.4f %.4f %.6f\n", left, right, step, result);
                }
            }
        } catch (Exception e) {
            System.out.println("Integration error: " + e.getMessage());
        }
    }
}

```

Рис. 10

```

public static void simpleThreads() {
    System.out.println("=== ПРОСТАЯ МНОГОПОТОЧНАЯ ВЕРСИЯ ===");

    Task task = new Task( tasksCount: 110);

    // Создаем потоки
    Thread generator = new Thread(new SimpleGenerator(task));
    Thread integrator = new Thread(new SimpleIntegrator(task));

    // Запускаем потоки
    generator.start();
    integrator.start();

    try {
        // Ждем завершения потоков
        generator.join();
        integrator.join();
    } catch (InterruptedException e) {
        System.out.println("Main thread interrupted");
    }

    System.out.println();
}

```

Рис. 11

```
Main x
Source 78,7837 105,2118 0,7132
Source 58,1266 169,9023 0,3879
Source 94,1200 140,4335 0,2133
Source 48,4087 138,5624 0,8059
Source 74,9869 102,6808 0,5561
Source 63,7990 150,8185 0,6455
Source 61,7763 159,3377 0,9477
Source 11,5055 136,0112 0,2463
Source 23,9425 118,2729 0,2988
Source 50,8932 155,6528 0,4157
Source 25,4317 141,7555 0,3816
Source 93,9874 145,6155 0,7486
Source 72,5269 154,8572 0,7530
Source 21,0714 183,8143 0,3724
Source 45,7331 118,6068 0,3226
Source 5,5188 168,1497 0,1216
Source 22,3422 165,1629 0,6967
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
Result 22,3422 165,1629 0,6967 366,284162
```

Рис. 12

```

package threads;

//класс для управления состоянием между потоками
6 usages
public class TaskState {

    2 usages
    private volatile boolean dataReady = false;

    2 usages
    private volatile boolean processing = false;

    2 usages
    private volatile int tasksGenerated = 0;

    2 usages
    private volatile int tasksProcessed = 0;

    1 usage
    public boolean isDataReady() {return dataReady;}

    2 usages
    public void setDataReady(boolean dataReady) {this.dataReady = dataReady;}

    2 usages
    public boolean isProcessing() {return processing; }

    2 usages
    public void setProcessing(boolean processing) { this.processing = processing;}

    no usages
    public int getTasksGenerated() {return tasksGenerated;}

    1 usage
    public void incrementTasksGenerated() {tasksGenerated++;}

    1 usage
    public int getTasksProcessed() {return tasksProcessed;}

    1 usage
    public void incrementTasksProcessed() {tasksProcessed++;}

}

```

Рис. 13

```

Main.java x SimpleGenerator.java x TaskState.java x SimpleIntegrator.java x Task.java x Functi
6 usages
private Task task;
5 usages
private TaskState state;

1 usage
public SimpleGenerator(Task task, TaskState state) {
    this.task = task;
    this.state = state;
}

@Override
public void run() {
    for (int i = 0; i < task.getTasksCount(); i++) {
        //ждём завершения обработки предыдущего задания
        while (state.isProcessing()) {
            Thread.yield();
        }

        //генерируем случайные параметры
        double base = 1 + Math.random() * 9;
        double left = Math.random() * 100;
        double right = 100 + Math.random() * 100;
        double step = Math.random();

        //устанавливаем параметры в задание
        task.setFunction(new Log(base));
        task.setLeftX(left);
        task.setRightX(right);
        task.setDiscretizationStep(step);

        System.out.printf("Source %.4f %.4f %.4f\n", left, right, step);

        //устанавливаем состояние
        state.setDataReady(true);
        state.setProcessing(true);
        state.incrementTasksGenerated();
    }
}

```

Рис. 14


```

Main.java x SimpleGenerator.java x TaskState.java x SimpleIntegrator.java x Task.java x Functions
private Task task;
7 usages
private TaskState state;

1 usage
public SimpleIntegrator(Task task, TaskState state) {
    this.task = task;
    this.state = state;
}

@Override
public void run() {
    try {
        while (state.getTasksProcessed() < task.getTasksCount()) {
            //активное ожидание данных
            if (state.isDataReady() && state.isProcessing()) {
                //получение и обработка данных
                double left = task.getLeftX();
                double right = task.getRightX();
                double step = task.getDiscretizationStep();

                double result = Functions.integrate(task.getFunction(), left, right, step);

                System.out.printf("Result %.4f %.4f %.4f %.6f\n", left, right, step, result);

                //сбрасываем состояние
                state.setDataReady(false);
                state.setProcessing(false);
                state.incrementTasksProcessed();
            } else {
                //если данные не готовы - ждём
                Thread.yield();
            }
        }
    } catch (Exception e) {
        System.out.println("Integration error: " + e.getMessage());
    }
}
}

```

Рис. 15

```

1 usage
public static void simpleThreads() {
    System.out.println("-----Простая многопоточная версия-----");

    //создаём общие объекты (задания и состояния) и устанавливаем количество заданий
    Task task = new Task( tasksCount: 110);
    TaskState state = new TaskState();

    //создаём потоки с общим состоянием
    Thread generator = new Thread(new SimpleGenerator(task, state));
    Thread integrator = new Thread(new SimpleIntegrator(task, state));

    //запускаем потоки
    generator.start();
    integrator.start();

    try {
        //ждём завершения потоков
        generator.join();
        integrator.join();
    } catch (InterruptedException e) {System.out.println("Главный поток был прерван!");}
    System.out.println();
}

```

Рис. 16

```
-----Простая многопоточная версия-----
Source 36,9571 143,8175 0,5057
Result 36,9571 143,8175 0,5057 206,229633
Source 35,6556 163,7010 0,4109
Result 35,6556 163,7010 0,4109 386,600874
Source 39,1878 136,8329 0,7345
Result 39,1878 136,8329 0,7345 223,483194
Source 35,1426 121,3250 0,5212
Result 35,1426 121,3250 0,5212 179,091368
Source 26,3177 105,9221 0,1574
Result 26,3177 105,9221 0,1574 155,839666
Source 42,9456 110,3123 0,3320
Result 42,9456 110,3123 0,3320 137,576675
Source 64,3298 165,6287 0,0916
Result 64,3298 165,6287 0,0916 217,254454
Source 95,7455 172,4838 0,1505
Result 95,7455 172,4838 0,1505 402,208094
Source 0,4598 154,2898 0,5501
Result 0,4598 154,2898 0,5501 316,928404
Source 43,8582 172,1500 0,2218
Result 43,8582 172,1500 0,2218 323,611880
Source 66,9153 144,2712 0,0013
Result 66,9153 144,2712 0,0013 158,484459
Source 69,6297 117,6279 0,1653
Result 69,6297 117,6279 0,1653 144,395509
```

Рис. 17

```
Source 70,0161 116,0897 0,2260
Result 70,0161 116,0897 0,2260 127,733291
Source 69,9337 154,8240 0,6250
Result 69,9337 154,8240 0,6250 545,705588
Source 36,0630 137,7183 0,7879
Result 36,0630 137,7183 0,7879 204,898796
Source 12,7026 113,9254 0,5481
Result 12,7026 113,9254 0,5481 2342,089251
Source 99,8821 111,1516 0,2273
Result 99,8821 111,1516 0,2273 35,228193
Source 62,2231 168,1359 0,8960
Result 62,2231 168,1359 0,8960 405,118570
Source 12,6981 171,9637 0,4638
Result 12,6981 171,9637 0,4638 339,521818
Source 22,9029 152,5362 0,9878
Result 22,9029 152,5362 0,9878 827,777376
Source 74,8354 191,3048 0,1184
Result 74,8354 191,3048 0,1184 437,058821
Source 35,6776 134,2862 0,6520
Result 35,6776 134,2862 0,6520 263,310904
Source 6,7070 160,4131 0,7496
Result 6,7070 160,4131 0,7496 441,545804

Process finished with exit code 0
```

Рис. 18

Задание 4

Причина незавершенных заданий в интегрирующем потоке в том, что интегрирующий поток не успевает обработать все сгенерированные задания, потому что потоки работают независимо и не координируют свое завершение. Для исправления нужно добавить явный сигнал завершения, чтобы интегратор знал, когда генератор закончил работу и можно завершать обработку оставшихся заданий.

Как оказалось после прочтения информации о семафорах, это можно было сделать и без дополнительного пользовательского класса (воспользовавшись, как раз – таки, семафором). Поэтому теперь реализуем два новых класса – `Generator` и `Integrator` – уже с применением одноместного семафора (Рис 19, 20 и рис. 21).

Для работы с ним используем специальные методы:

1. `!isInterrupted()` – проверяем, не сказали ли нам остановиться. Это важно для корректного завершения. По сути, `interrupt` – это, если можно так сказать, флаг, обозначающий, что поток нужно прервать;
2. `acquire()` – блокирует поток, пока не получит разрешение;
3. `release()` – освобождает семафор, чтобы другой поток мог начать работу.

А также добавляем `try – catch`, в котором ловим специальное исключение `InterruptedException` – означает, что выполнение текущего потока было прервано извне.

Добавляем в `Main` третий метод – `complicatedThreads()` (Рис 22). `Semaphore(1, true)` – внутри этого метода создаем семафор с 1 разрешением и честной очередью(FIFO). Также в `complicatedThreads()` оба созданных нами потока получают ссылки на один и тот же объект `task` и один и тот же `semaphore` (поскольку они должны работать с одними и теми же данными). Запускаем потоки с помощью метода `start()`, который создает новый поток и запускает в нем метод `run()`. Главный поток (`main`) засыпает на 50 мс. В это время наши потоки `generator` и `integrator` работают. Через 50 мс главный поток прерывает оба потока, потоки корректно завершаются, главный поток продолжает.

После нескольких запусков программы всё работает корректно (Рис. 23 и рис. 24).


```
Main.java × Generator.java × Integrator.java × SimpleGenerator.java × TaskState.java ×
package threads;
import java.util.concurrent.Semaphore;

2 usages
public class Generator extends Thread {
    6 usages
    private Task task;
    3 usages
    private Semaphore semaphore;

    1 usage
    public Generator(Task task, Semaphore semaphore) {
        this.task = task;
        this.semaphore = semaphore;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < task.getTasksCount() && !isInterrupted(); i++) {
                //запрашиваем разрешение на запись
                semaphore.acquire();

                //генерируем случайные параметры
                double base = 1 + Math.random() * 9; // от 1 до 10
                double left = Math.random() * 100; // от 0 до 100
                double right = 100 + Math.random() * 100; // от 100 до 200
                double step = Math.random(); // от 0 до 1
            }
        }
    }
}
```

Рис. 19

```
//устанавливаем параметры в задание
task.setFunction(new functions.basic.Log(base));
task.setLeftX(left);
task.setRightX(right);
task.setDiscretizationStep(step);

System.out.printf("Source %.4f %.4f %.4f\n", left, right, step);

//освобождаем семафор для чтения
semaphore.release();
}
} catch (InterruptedException e) {System.out.println("Generator interrupted");}
}
```

Рис. 20

```

package threads;
import java.util.concurrent.Semaphore;
import functions.Functions;

2 usages
public class Integrator extends Thread {
    6 usages
    private Task task;
    3 usages
    private Semaphore semaphore;

    1 usage
    public Integrator(Task task, Semaphore semaphore) {
        this.task = task;
        this.semaphore = semaphore;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < task.getTasksCount() && !isInterrupted(); i++) {
                //запрашиваем разрешение на чтение
                semaphore.acquire();

                //получаем параметры задания
                double left = task.getLeftX();
                double right = task.getRightX();
                double step = task.getDiscretizationStep();

                //вычисляем интеграл и выводим результат
                double result = Functions.integrate(task.getFunction(), left, right, step);
                System.out.printf("Result %.4f %.4f %.4f %.6f\n", left, right, step, result);

                //освобождаем семафор для записи
                semaphore.release();
            }
        }
        catch (InterruptedException e) {System.out.println("Integrator interrupted");}
        catch (Exception e) {System.out.println("Integration error: " + e.getMessage());}
    }
}

```

Рис. 21

```
1 usage
public static void complicatedThreads() {
    System.out.println("-----Усложнённая многопоточная версия-----");

    //создаём общие объекты (задания и семафора) и устанавливаем количество заданий
    Task task = new Task( tasksCount: 130);
    Semaphore semaphore = new Semaphore( permits: 1, fair: true);

    //создаем потоки
    Generator generator = new Generator(task, semaphore);
    Integrator integrator = new Integrator(task, semaphore);

    //запускаем потоки
    generator.start();
    integrator.start();

    try {
        Thread.sleep( millis: 50);

        //прерываем потоки
        generator.interrupt();
        integrator.interrupt();

        //ждем завершения потоков
        generator.join();
        integrator.join();
    } catch (InterruptedException e) {System.out.println("Главный поток был прерван!");}
    System.out.println();
}
```

Рис. 22

```
Main x
-----Усложнённая многопоточная версия-----
Source 57,1574 192,5761 0,1589
Result 57,1574 192,5761 0,1589 501,441712
Source 60,7448 149,4198 0,3428
Result 60,7448 149,4198 0,3428 195,858080
Source 54,1580 161,5496 0,4431
Result 54,1580 161,5496 0,4431 238,044209
Source 97,0166 159,1366 0,3573
Result 97,0166 159,1366 0,3573 142,939770
Source 68,5369 156,3166 0,9570
Result 68,5369 156,3166 0,9570 203,900870
Source 8,5332 138,0841 0,5056
Result 8,5332 138,0841 0,5056 319,375993
Source 5,3317 168,9443 0,9168
Result 5,3317 168,9443 0,9168 526,618818
Source 38,6069 172,2753 0,4569
Result 38,6069 172,2753 0,4569 1796,501821
Source 15,7201 188,8281 0,4415
Result 15,7201 188,8281 0,4415 6699,439958
Source 23,8252 160,3862 0,4545
Result 23,8252 160,3862 0,4545 719,688985
Source 56,8312 100,3314 0,6265
Result 56,8312 100,3314 0,6265 207,271273
Source 66,2432 106,7110 0,5485
Result 66,2432 106,7110 0,5485 79,952535
Source 27,9792 145,3710 0,4691
Result 27,9792 145,3710 0,4691 229,105323
```

Рис. 23

```
Main x
Result 50,0685 183,3925 0,3100 276,481085
Source 5,9096 102,9340 0,5502
Result 5,9096 102,9340 0,5502 169,046570
Source 13,2929 162,0202 0,1907
Result 13,2929 162,0202 0,1907 445,960682
Source 32,7120 142,4684 0,8737
Result 32,7120 142,4684 0,8737 218,722071
Source 48,9415 106,0272 0,8569
Result 48,9415 106,0272 0,8569 207,505332
Source 93,9725 135,8229 0,2487
Result 93,9725 135,8229 0,2487 90,006423
Source 74,5445 128,2933 0,7195
Result 74,5445 128,2933 0,7195 216,531243
Source 40,9965 199,2090 0,6713
Result 40,9965 199,2090 0,6713 398,887813
Source 33,4744 145,6085 0,4864
Result 33,4744 145,6085 0,4864 253,575269
Source 93,9749 161,8177 0,1369
Result 93,9749 161,8177 0,1369 368,425939
Source 38,2719 142,1990 0,1371
Result 38,2719 142,1990 0,1371 206,197075
Source 40,4634 115,8724 0,5449
Result 40,4634 115,8724 0,5449 317,472778

Process finished with exit code 0
```

Рис. 24